

Data Flow Diagrams and Database Design



Data Flow Diagrams

- Purpose
 - Data flow diagrams are a diagrammatic representation of the information flows within a system:
 - How information enters and leave the system?
 - What changes the information?
 - Where information is stored?
 - Applications in SA&D
 - Boundary definition
 - Checking the completeness of analysis
 - Basis for program specification



Data Flow Diagrams

- In SSADM, four sets of DFD will be developed
 - Current Physical
 - Current Logical
 - Business System Options
 - Required System



Data Flow Diagrams

- External source / recipients (external entities)
 - Provide information to / receive information from the system
- Process
 - A process transforms data with the system
 - Identifier, location and
 - name





Data Flow Diagrams

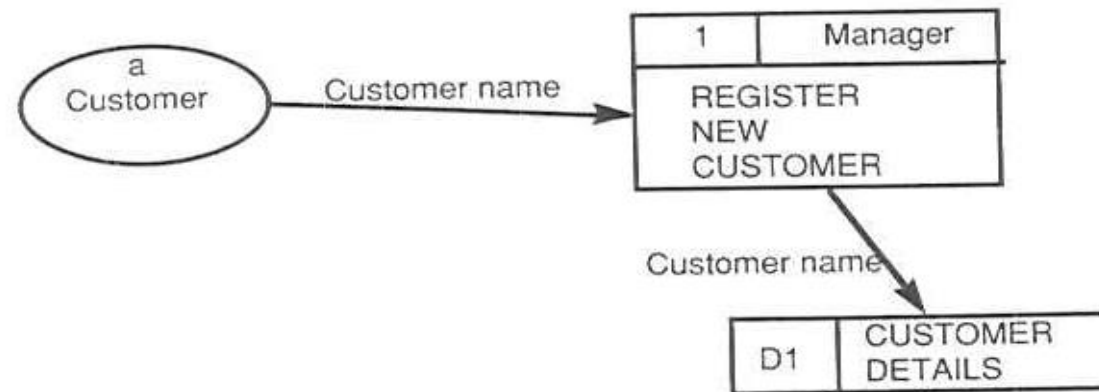
- Data Store

- A data store is where information is held for a time within the system



- Data Flow

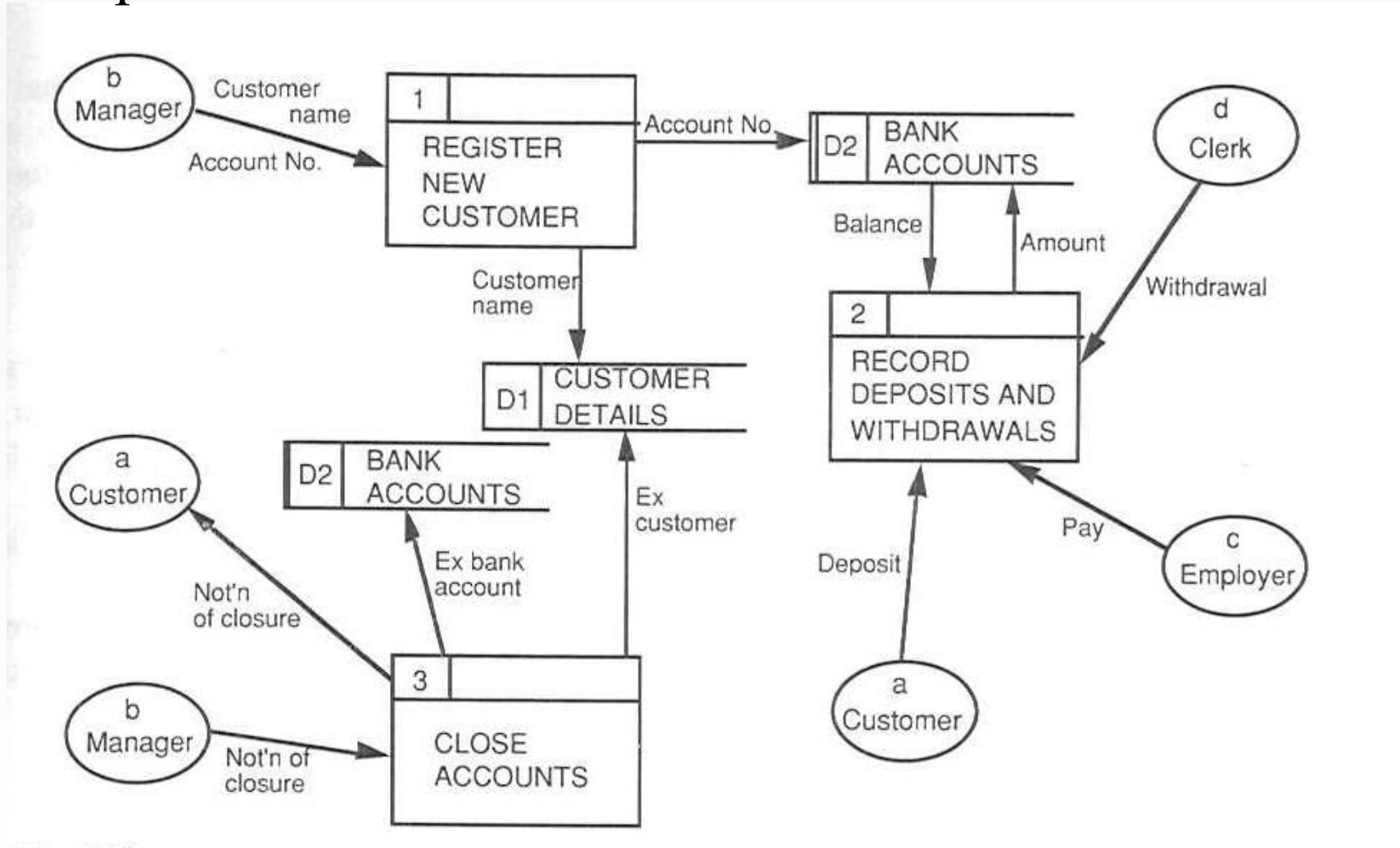
- A package of information flowing between objects
- Data flow only to / from a process, the other end can be external source or data store





Data Flow Diagrams

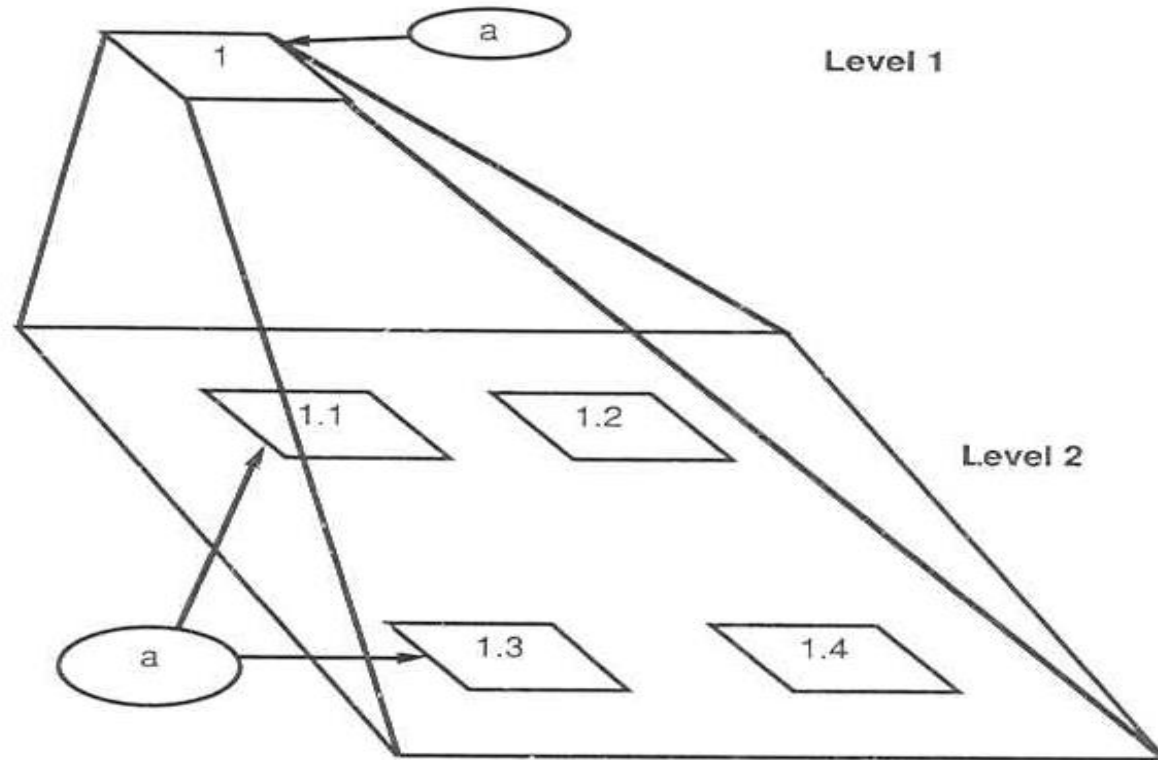
- An example





Data Flow Diagrams

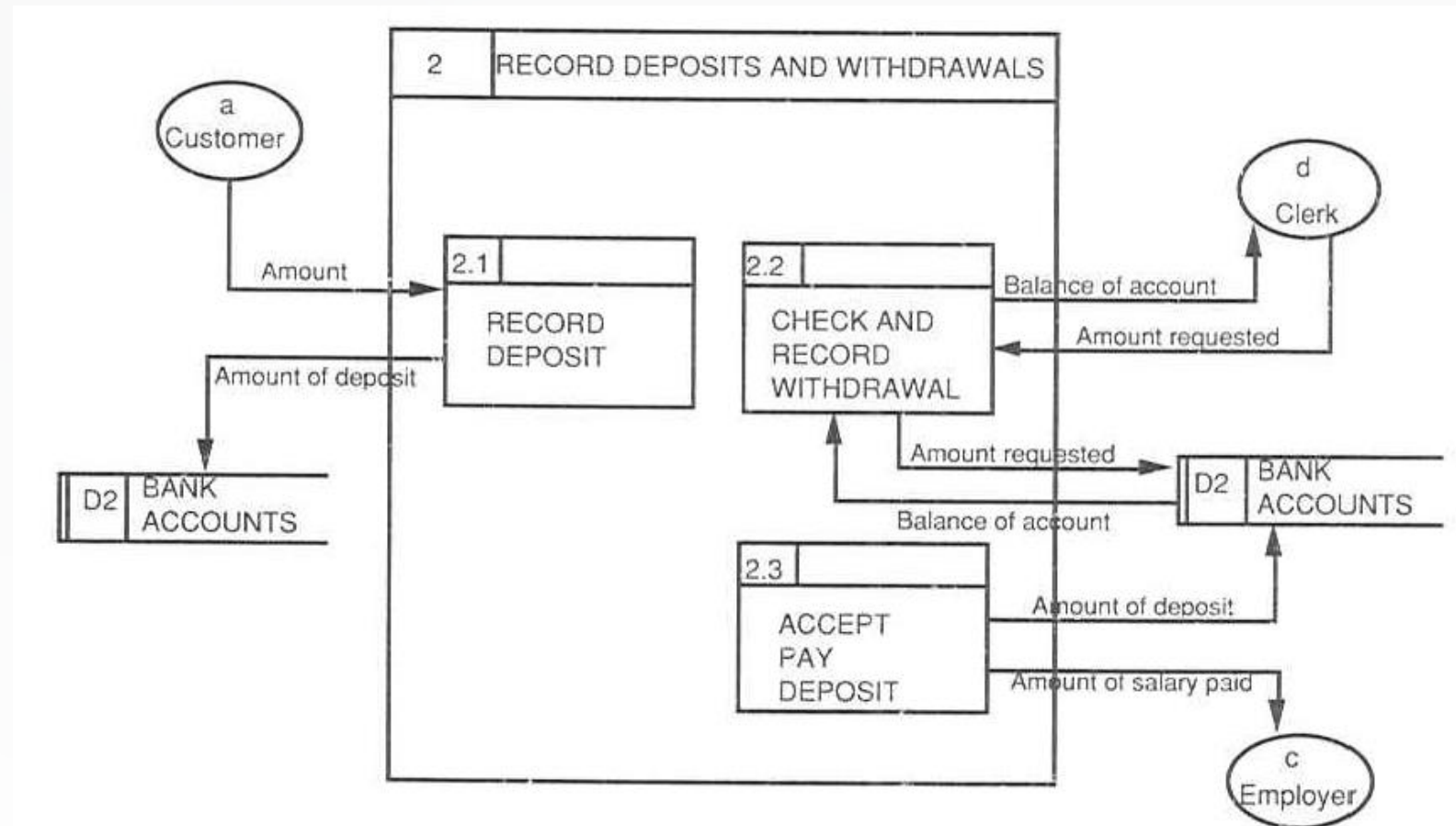
- Levels of data flow diagrams
 - Each process on DFD can be broken down into several processes which are shown on another DFD.





Data Flow Diagrams

- Levels of data flow diagrams
 - An example:



Database



What is Database?

Data + Base

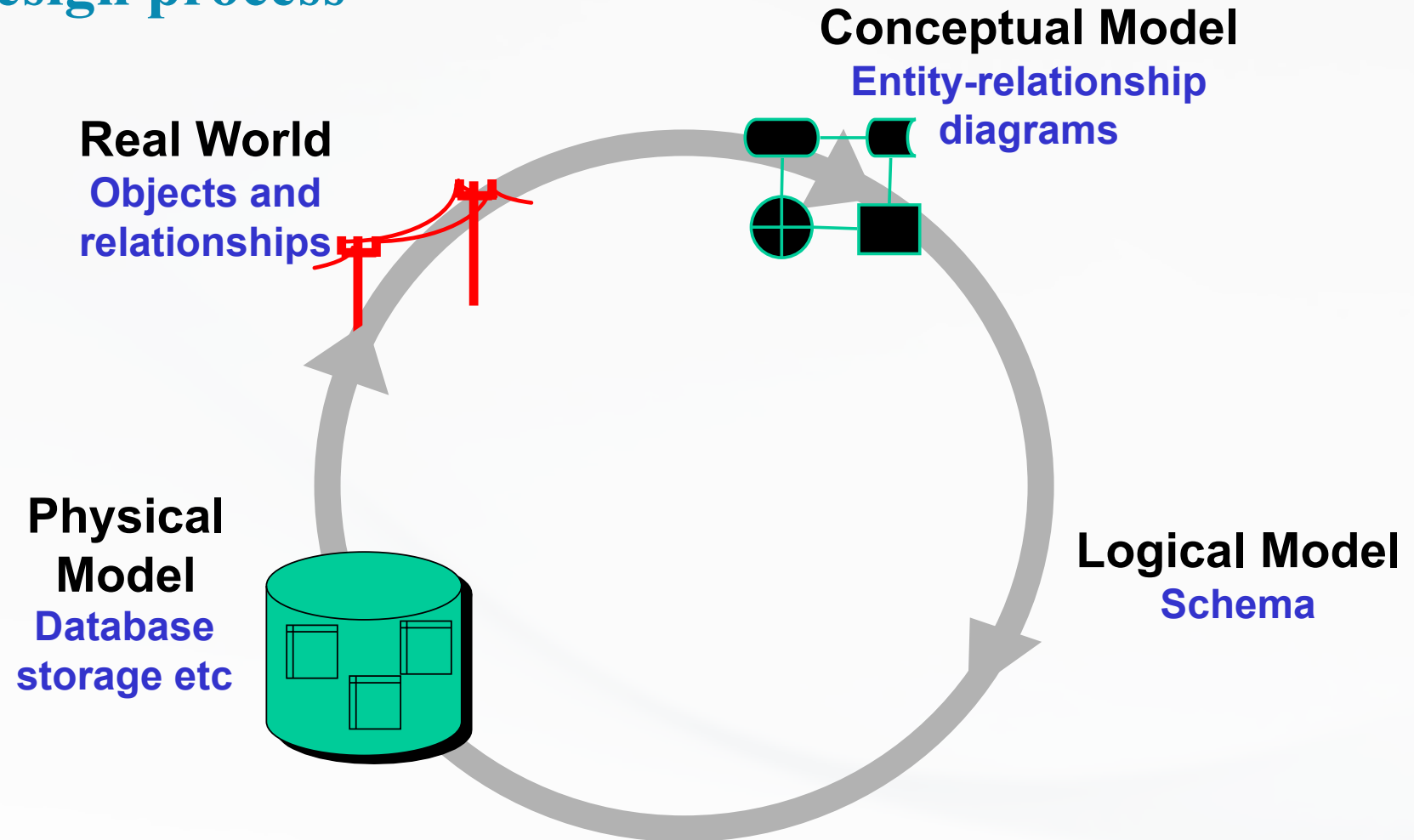
x_y.txt		
<i>Customer-ID</i>	<i>X-coord</i>	<i>Y-coord</i>
1	-122.43294	37.77458
2	-122.42018	37.77026
3	-122.41448	37.77869
4	-122.39671	37.76638
5	-122.39398	37.77390
6	-122.42793	37.76501
7	-122.40856	37.77458

Addresses.dbf	
<i>Customer-ID</i>	<i>Address</i>
8	96 Anson PL.
9	252 Jessie St.
10	2305 Webster St.
11	327 Kearny St.
12	2604 Jackson St.
13	1030 Lombard St.
14	72 Market St.
15	102 Marina Blvd.



Database Design

Database design process





Database Design

Database applications are modeled using a three-step design process

Conceptual data modelling (ER model)



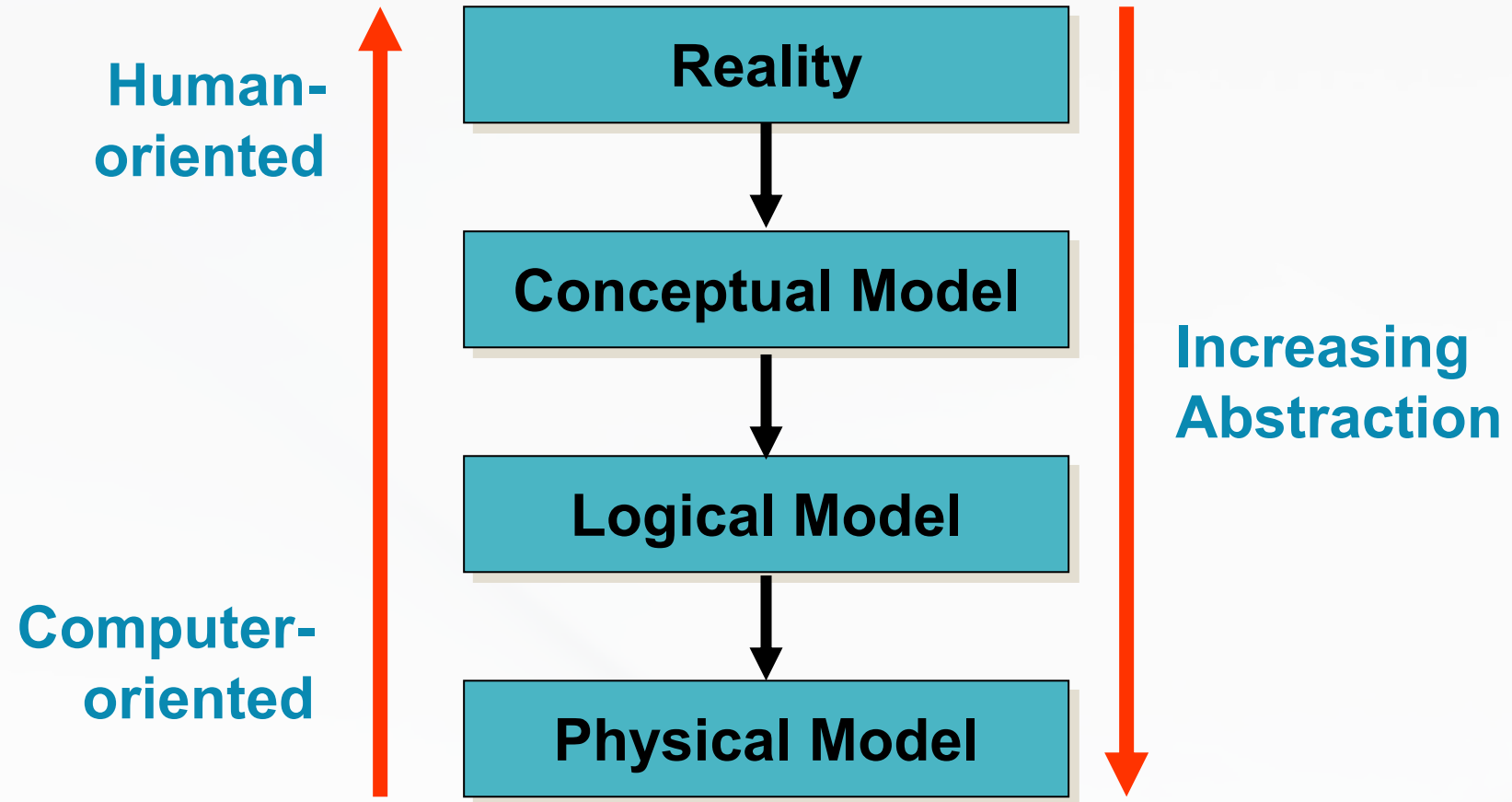
Logical-mapping to a Relational model
and associated query language



Physical-file structures, indexing



Data Model Levels





Application Domain

- Spatial application domain
 - A *state-park* consists of *forests*
 - A forest is a collection of *forest-stands* of different species
 - State-Park is accessed by *roads* and has a *manager*
 - State-Park has *facilities*
 - *River* runs through state-park and supplies water to the facilities

Entity Relationship Diagram



Conceptual DM: The ER Diagram



What is ER Diagram?

An Entity Relationship Diagram is a visual representation of different data using **conventions** that describe how these data are related to each other.

- 3 basic concepts
 - **Entities** have an independent conceptual or physical existence.
 - Examples: Forest, Road, Manager, ...
 - Entities are characterized by **Attributes**
 - Example: Forest has attributes of name, elevation, etc.
 - An Entity interacts with another Entity through **relationships**.
 - Road allow access to Forest interiors.
 - This relationship may be named “Accesses”



Relationship Types

- Relationships can be categorized by
 - **Binary relationship**: two entities participate
 - **Cardinality** constraints: enterprise rules
- Types of Cardinality constraints for binary relationships
 - **One-One**: An instance of an entity relates to a unique instance of other entity.
 - **Many-One**: Many instances of an entity relate to an instance of an other.
 - **Many-Many**: Many instances of one entity relate to multiple instances of another.
- Exercise: Identify type of cardinality constraint for following:
 - Many facilities belong to a forest. Each facility belong to one forest.
 - A manager manages 1 forest. Each forest has 1 manager.
 - A river supplies water to many facilities. A facility gets water from many rivers.

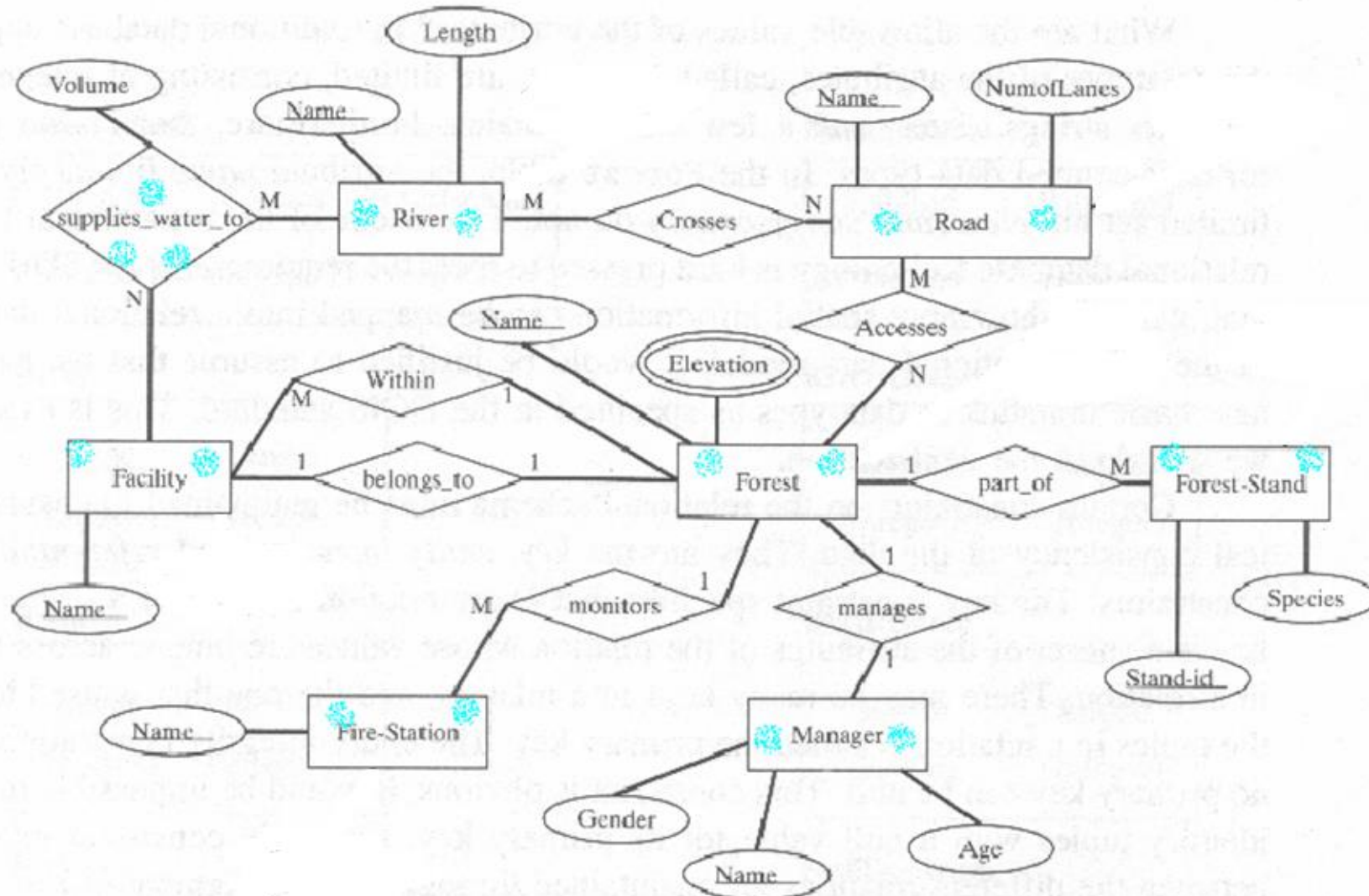


ER Diagrams Geographical Notation

Concept	Symbol
Entities	
Attributes	
Multi-valued Attributes	
Relationships	
Cardinality of Relationship	1:1, M:1, M:N



ER Diagram for 'State-Park'





Logical Data Model: The Relational Model

- Relational model is based on set theory
- Main concepts
 - Domain: a set of values for a simple attribute
 - Relation: cross-product of a set of domains
 - Represents a table, i.e. homogeneous collection of rows (tuples)
 - The set of columns (i.e. attributes) are same for each row



Mapping ER to Relational

- Translation rules
 - **Entity** becomes Relation
 - **Attributes** become columns in the relation
 - **Multi-valued attributes** become a new relation
 - includes foreign key to link to relation for the entity
 - **Relationships** (1:1, 1:N) become foreign keys
 - if the cardinality of the relationship is N:1, foreign key in the relation is on the N-side
 - **M:N Relationships** become a relation (key is the primary keys of the participating parties)



Relational Schema

Forest-Stand

<u>Stand-id</u>	Species	Forest-name	<u>Polygonid</u>
(Integer)	(varchar)	(varchar)	(Integer)

River

<u>Name</u>	Length	<u>Lineid</u>
(varchar)	(Real)	(Integer)

Road

<u>Name</u>	NumofLanes	<u>Lineid</u>
(varchar)	(Integer)	(Integer)

Facility

<u>Name</u>	Forest-name	Forest-name-2	<u>Pointid</u>
(varchar)	(varchar)	(varchar)	(Integer)



Relational Schema (cont'd)

Forest

<u>Name</u>	<u>Polygonid</u>
(varchar)	(Integer)

Fire-Station

<u>Name</u>	ForName	<u>Pointid</u>
(varchar)	(varchar)	(Integer)

Supplies_Water_To

<u>FacName</u>	<u>RivName</u>	Volume
(varchar)	(varchar)	(Real)

Road-Access-Forest

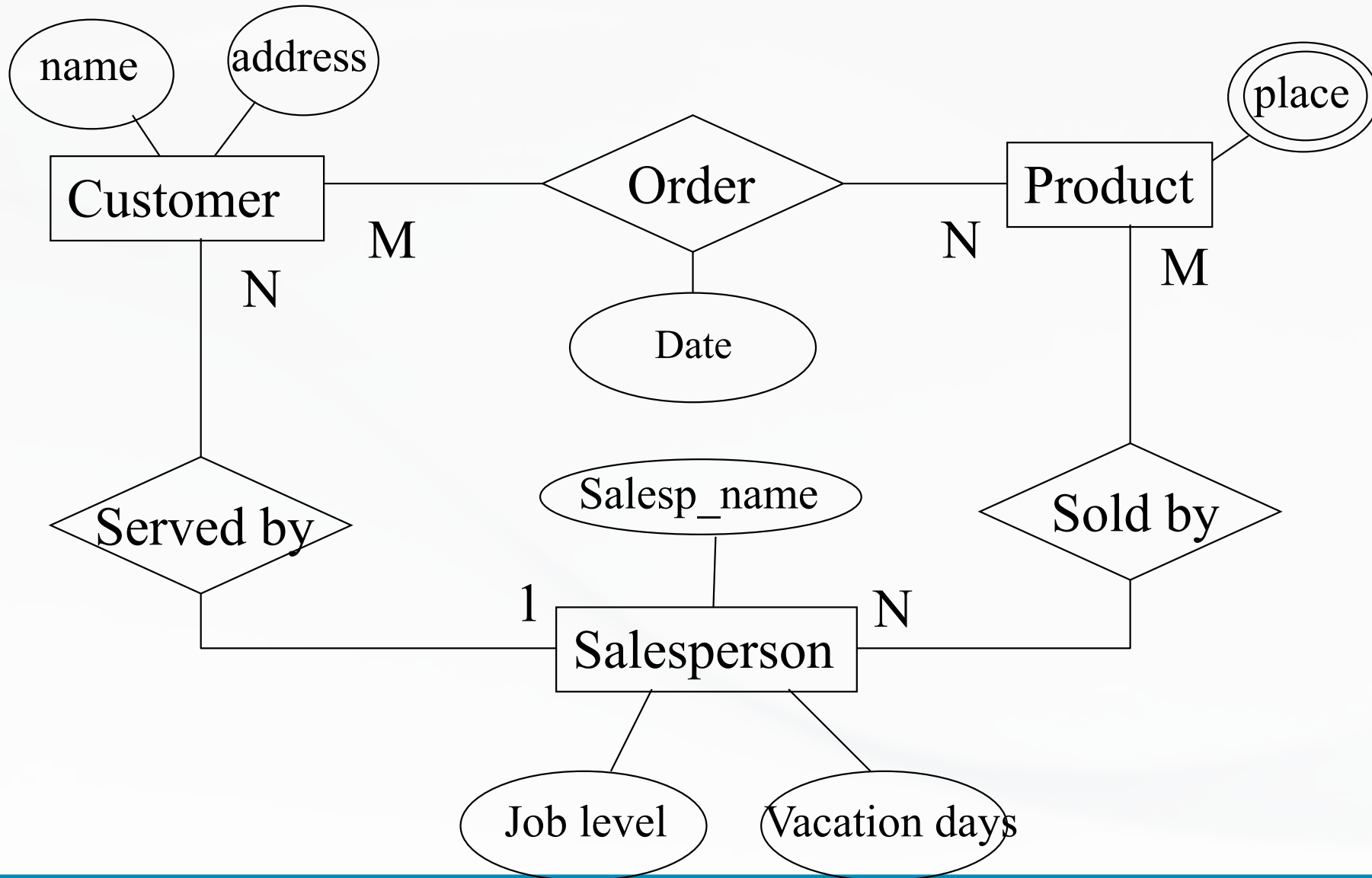
<u>RoadName</u>	<u>ForName</u>
(varchar)	(varchar)

Manager

<u>Name</u>	<u>Age</u>	<u>Gender</u>	ForName
(varchar)	(Integer)	(varchar)	(varchar)



Another Example





Relational Schema

Custm_name	address
------------	---------

Product_name	place
--------------	-------

Sold_by

Product_name	Salesp_name
--------------	-------------

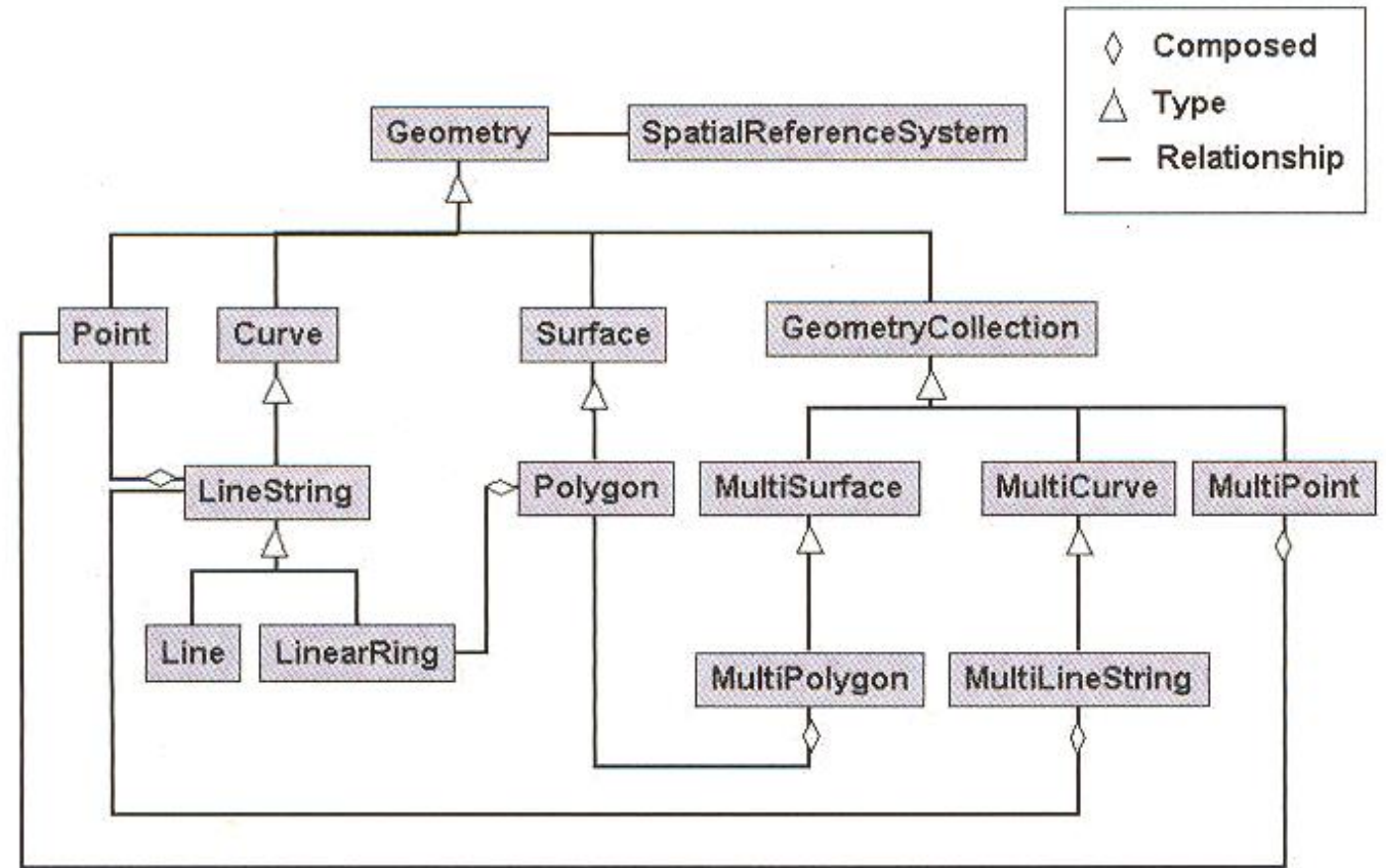
Salesp_name	joblevel	vacationdays
-------------	----------	--------------

Order_id	date	Custm_name	Product_name
----------	------	------------	--------------



Object-Relationship Model

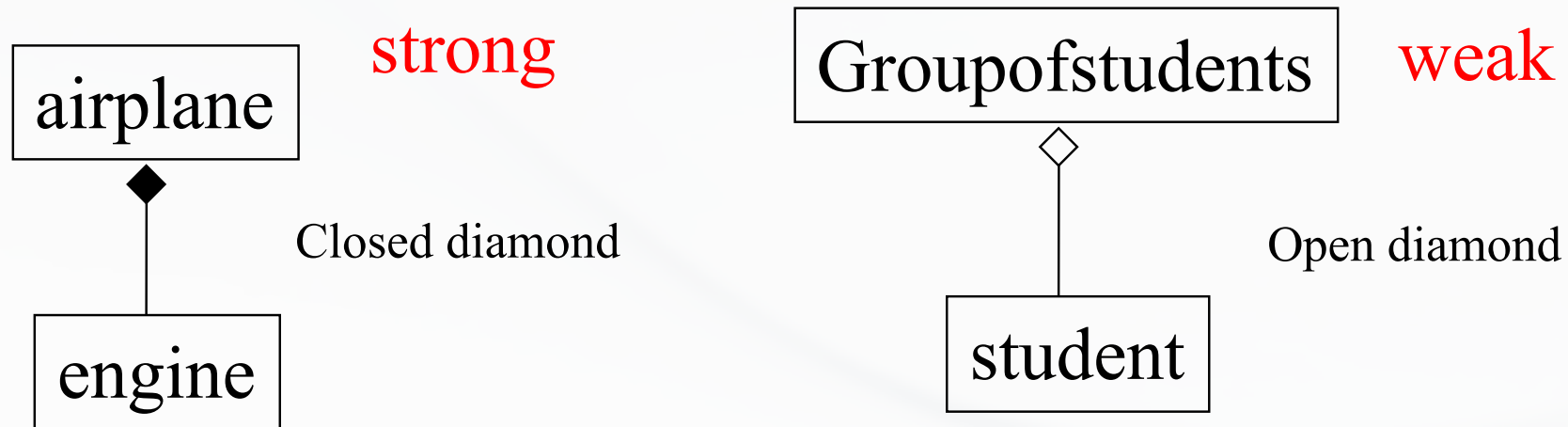
- Relationship
 - Aggregation:
 - part-whole
 - Generalization:
 - is_a
 - Association
 - Multiplicities on two ends





Relationships

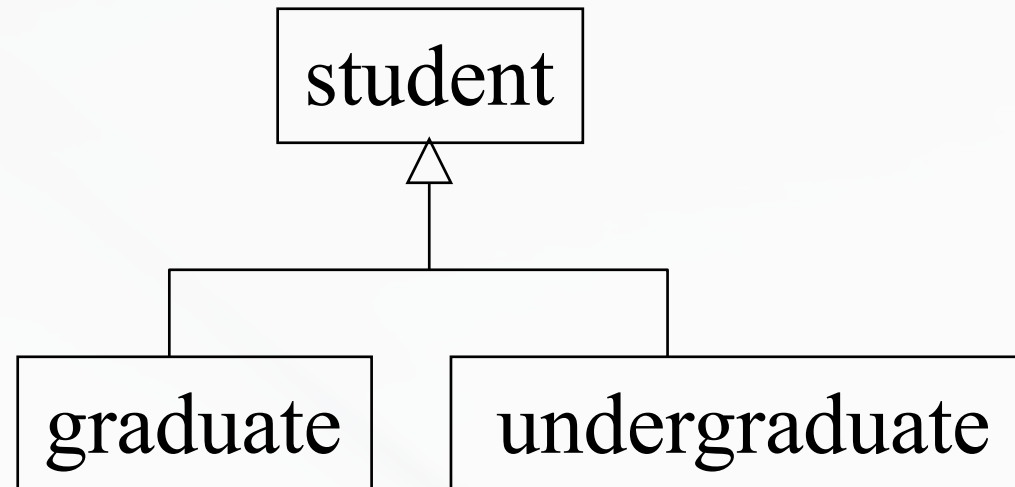
- **Aggregation:** represents the part-whole relationship.
 - Strong: parts may not be the only ones to compose another object or an object cannot exist in a functionally correct form if one of its component parts is missing
 - Weak: a collection of similar elements





Generalization

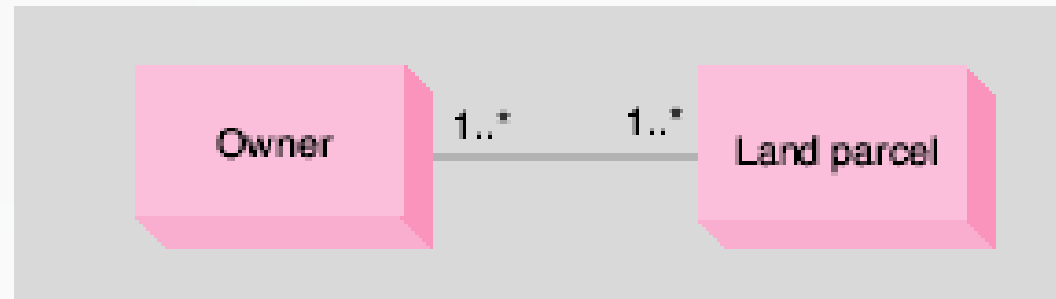
- **Generalization:** defines specialized classes that share properties and methods with the superclass and have additional properties and methods.





Relationships (cont'd)

- **Association:** represents relationships between different classes. They have defined multiplicities at both ends.





Multiplicity

A ***Multiplicity*** is a constraint on the number of objects that can be associated with another object.

Notation for multiplicities:

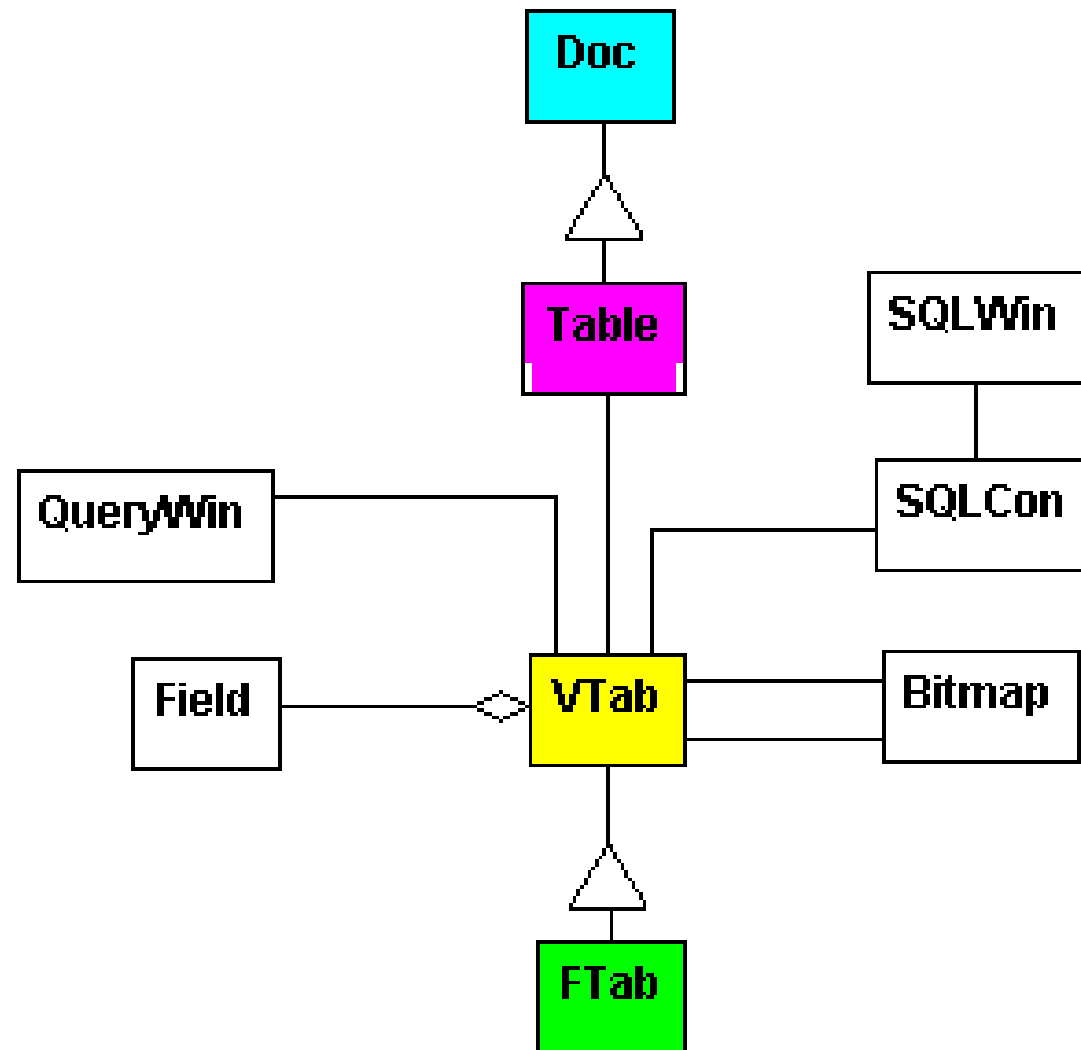
1 - One and only one. Showing this multiplicity is optional; if none is shown, “1” is implied.

0..1 - Zero or one

M..N - From M to N (positive integers)

* or 0..* - From zero to any positive integer

1..* - From one to any positive integer





Object-Relationship Model Example

